Experiments and manuscript updates to complete

Highest-priority experiments

- [x] **Attribute-aware baselines under the same information budget.** The matched five-seed ImageNet10 comparison is complete for exact target-slice plus stratified background sampling, Attribute-Filtered Random, and Attribute-Filtered Entropy.
- [x] **End-to-end cost accounting.** Report source-pool attribute assignment, repair-image generation, generated-image attribute assignment, subset selection, and retraining time separately. Distinguish one-time task preprocessing from per-slice and per-repair costs.
- [ ] **Attribute-generation ablation.** On a representative setting, rerun selection with an alternative attribute source or annotator under the same closed-set vocabulary and repair protocol. Compare attribute agreement, selected-set overlap, target-slice enrichment, and downstream repair performance with the main Qwen3-VL assignment.
- [x] **Slice-granularity sensitivity.** Five-seed COCO and ImageNet10 comparisons now cover single-, two-, and three-attribute repair sets over source ratios 0.1--0.9.
- [ ] **Vocabulary-coverage sensitivity.** Controlled ImageNet10 and COCO missing-field diagnostics are complete, but the merged-value transformation remains unrun.
- [x] **Alternative-pipeline response integration.** The verified InstructPix2Pix and ICEdit results are included in the KaLa response.

Additional measurements to insert

- [ ] Measure the missing retraining wall-clock times and the ImageNet10 source-annotation time.
- [ ] Insert target-slice gains and non-target performance changes next to the overall task metrics.

Response to Area Chair 6k27

Dear Area Chair 6k27, thank you for summarizing the reviewers' main concerns. We address the evaluation scope and the limits of our claims below.

> Reviewers yyPZ and hG6j raise the concern of the end-to-end cost and misleading claims about efficiency. It is also related to another shared concern about fair comparison against baselines. As stated in the above strengths section, it is very reasonable to leverage the labeled attributes since the upstream provides them. However, it seems that none of the compared baselines use these attributes.Given attributes, the proposed method is actually not restricted to model repair, but it can be used in any data selection scenario. The difference is that the upstream has provided attribute labels in this paper, while for standard data selection, attribute generation will usually be counted into the method's computation. The authors ingeniously pick this setting so that it cannot be strictly blamed for discarding attribute generation when calculating computational overhead. While the reported efficiency cannot be directly criticized, I do agree that the adopted baselines should include methods considering the generated attribute labels. The authors should accordingly include attribute-aware baselines as suggested by the reviewers. Otherwise, this is merely a standard data selection method that should be evaluated outside of the model repair scenario. It would further strengthen the paper if the authors indeed include such comparisons on standard data selection scenarios. Another possibility is that the compared baselines should also use the attribute values instead of extracting embeddings. This will make the efficiency comparison fairer.

> Reviewers hG6j and KaLa raise the question regarding different repair pipelines. Since AttriSelect itself only involves selecting from the source pool and generated data, it would be stronger if the authors could evaluate the model against multiple pipelines. The authors should clarify the implementation details regarding the entire repair pipeline. When the ablation on generative models has been provided, it would further strengthen the paper if the authors could also ablate the attribute generation and other pipeline components. The robustness against imperfect attribute vocabularies will also be important for the pipeline.

Response. We added a source-only ImageNet10 evaluation outside model repair. This setting uses no generated images, diagnosed slice, or repair checkpoint; each selected subset trains a fresh ResNet-18. Both tables report five-seed mean ImageNet10 test accuracy (%).

We also include three attribute-aware baselines that were not evaluated in the main paper: Attribute Rarity, Semantic CoreSet, and Object--Attribute Entropy Reduction [2].

Attribute Rarity. We score each image by the class-conditional rarity of

its VLM-assigned attribute values and apply soft class balancing. In Active Learning, rarity counts use the labeled set and candidate classes use model predictions.

Semantic CoreSet. We one-hot encode the VLM-assigned attribute values, assign an exact class-balanced quota, and run greedy k-center selection within each class in this semantic representation. The implementation selects without replacement.

Object--Attribute Entropy Reduction [2]. Since the attributes are already known, we adapt the method to query only the unknown class label. It prioritizes samples with high model class entropy and under-labeled attribute values whose labeled class distributions remain uncertain, followed by soft class balancing.

Source Ratio	Random	Stratified	Entropy	CoreSet	BADGE	Attribute Rarity	Semantic CoreSet	A
0.1	43.23	41.85	26.86	38.33	42.62	44.46	44.08	
0.3	55.84	57.15	44.64	56.16	58.17	56.54	56.55	
0.5	64.77	64.14	58.69	64.87	63.19	65.31	62.56	
0.7	69.47	71.63	66.71	68.03	69.57	71.01	67.18	
0.9	71.34	71.70	72.88	72.89	72.38	72.48	71.24	

AttriSelect is strongest at all five one-shot budgets, with gains of `0.03--1.22` accuracy points over the strongest baseline.

Ratio	Random	Entropy	CoreSet	BADGE	Attribute Rarity	Object--Attribute Entropy Reduction [2]	AttriSelect-AL
0.1	43.23	43.23	43.23	43.23	43.23	43.23	43.23

Ratio	Random	Entropy	CoreSet	BADGE	Attribute Rarity	Object--Attribute Entropy Reduction [2]	AttriSelect-AL
0.2	51.11	**52.26**	50.00	51.27	50.93	50.48	51.33
0.3	55.38	54.85	55.01	56.17	52.94	55.95	**56.96**
0.4	58.71	58.63	59.54	60.46	60.47	60.31	**63.02**
0.5	61.97	63.70	63.92	61.89	61.86	63.88	**65.18**
0.6	66.15	66.70	68.16	67.11	65.78	65.79	**68.80**
Curve Mean	56.09	56.56	56.64	56.69	55.87	56.61	**58.09**

All Active Learning methods share the same initial labeled set at ratio `0.1`. **AttriSelect-AL is strongest at ratios `0.3--0.6` and improves the Curve Mean by `1.40` points over BADGE.** Under matched training and information settings, the same attribute-aware selection principle therefore extends to standard source-only data selection.

Attribute-source sensitivity. In a separate diagnostic, we reran AttriSelect with attributes assigned by Qwen3-VL (ours), BLIP, and InternVL3 while keeping the source-only training protocol fixed.

Attribute Source	One-Shot Mean	Active Learning Curve Mean	Final Accuracy at Ratio 0.6
Qwen3-VL (ours)	61.32	**57.04**	**64.77**
BLIP	61.18	55.95	64.46
InternVL3	**61.86**	56.33	64.23

No attribute source dominates all one-shot budgets, and their means differ

by at most `0.68` accuracy points. Qwen3-VL (ours) gives the strongest Active Learning Curve Mean and final accuracy. Thus, the attribute source affects individual results, but the method does not depend on one specific VLM.

References

1. SafeFix: Targeted Model Repair via Controlled Image Generation. TMLR, 2026.
2. Actively Selecting Annotations Among Objects and Attributes. ICCV, 2011.

Response to Reviewer yyPZ

Dear Reviewer yyPZ, thank you for raising these questions about end-to-end cost, comparison fairness, and the interpretation of FR.

Q1 — End-to-End Computational Cost

> "Misleading Claims Regarding Computational Efficiency. The authors highlight in Table 2 that AttriSelect takes less than 2 seconds, claiming a speedup of up to 283× over baselines. However, this entirely omits the massive computational overhead of the prerequisite step: running Qwen3-VL to annotate the entire source pool. Appendix A states it takes 10–20 minutes to annotate just 1,000 images. For datasets like CelebA (80,000 images) or COCO (20,000 images), this requires tens to hundreds of GPU hours. Claiming high efficiency while ignoring the cost of generating the metadata that the algorithm relies upon is highly misleading."

Accounting Scope

Response. Table 2 reports the cost of subset selection, not the complete repair pipeline, and the revision will make that scope explicit. The reported speedups compare online selector computation after attribute records are available. **They should not be interpreted as end-to-end speedups** over source annotation, repair-data construction, and retraining.

We report repair-image generation separately because **the generated repair set is a fixed, publicly available dataset shared by every selector**, not a set regenerated for each method. Every selector receives the same repair images as pre-existing augmentation data. Regenerating the SafeFix [1] set would add the same one-time dataset-construction cost to every method without changing their runtime ordering. The table columns therefore cover source-side preprocessing and selection through retraining and evaluation. By contrast, **we include Qwen3-VL source-pool assignment for AttriSelect** because it is a method-specific prerequisite.

Conservative Comparison

For a fixed task-level attribute vocabulary, **source-pool assignment is a one-time preprocessing cost, and its metadata can be reused** across source ratios, selectors, and later repair iterations on the same task. The full pool does not need to be annotated again for every repair. Here, **the first repair run assumes that source attributes must be assigned from scratch**. Its cost covers source preprocessing and selection through retraining and evaluation; repair-image generation is reported separately.
Even so, **every AttriSelect row below charges the full fresh-assignment cost**. On one NVIDIA H200 GPU, the mean over two complete 5,000-image KITTI runs is **32.77 minutes**. The earlier 10--20 minutes per 1,000 images was measured on an NVIDIA A5000; the H200 takes approximately 5 minutes per 1,000 images. Assigning the complete 20,000-image COCO source pool on the same H200 takes approximately **1.50 hours**.

KITTI: 5,000 source images. Each AttriSelect row charges the mean fresh 5,000-image H200 Qwen3-VL assignment time of 32.77 minutes.

Ratio	Random	Stratified	Entropy	CoreSet	BADGE	UHerding	
0.1	42.17 min	42.18 min	48.82 min	1.37 h	1.12 h	43.47 min	43.5
0.3	43.84 min	43.85 min	50.28 min	1.44 h	1.20 h	45.08 min	45.6
0.5	45.18 min	45.18 min	51.73 min	1.49 h	1.27 h	46.42 min	47.4
0.7	47.04 min	47.05 min	53.46 min	1.56 h	1.35 h	48.16 min	49.7

Ratio	Random	Stratified	Entropy	CoreSet	BADGE	UHerding	
0.9	48.86 min	48.87 min	55.53 min	1.63 h	1.42 h	50.34 min	51.4

COCO: 20,000 source images. Image generation and generated-repair-image assignment are excluded. Each entry includes the applicable source preprocessing and selection cost, retraining, and evaluation. The AttriSelect column charges a fresh full-pool Qwen3-VL assignment in every row.

Ratio	Random	Stratified	Entropy	CoreSet	BADGE	UHerding
0.1	4.41 min	4.42 min	10.98 min	16.18 min	19.63 min	5.08 min
0.3	8.18 min	8.18 min	14.74 min	39.86 min	49.76 min	8.93 min
0.5	12.37 min	12.36 min	19.04 min	1.08 h	1.56 h	13.08 min
0.7	15.73 min	15.75 min	22.27 min	1.45 h	2.29 h	16.34 min
0.9	19.66 min	19.65 min	26.28 min	1.87 h	3.01 h	20.18 min

Cost–Performance Interpretation

> **Cost takeaway.** Combined with the main results in the original paper, these tables show a clear cost--performance trade-off. **Random, Stratified, UHerding, and PPAL are faster, especially at small source ratios, but their repair performance is generally lower.** For example, at COCO ratio 0.1, AttriSelect takes 1.57 hours when fresh annotation is charged, whereas Random, UHerding, and PPAL take 4.41, 5.08, and 6.97 minutes, respectively; their test AP values are 0.6048, 0.6013, and 0.5950, compared with 0.6183 for AttriSelect.
>
> **CoreSet and BADGE are generally the two strongest baselines besides AttriSelect, but they become much slower at high source ratios.** At ratio 0.9, they are even slower than AttriSelect after the AttriSelect total conservatively includes VLM annotation of the entire source pool. On COCO, AttriSelect takes 1.72 hours and reaches 0.6355 AP, compared with 1.87 hours/0.6202 for CoreSet and 3.01 hours/0.6213 for BADGE. On KITTI, AttriSelect takes 1.15 hours and reaches 0.5850 mAP50--95, compared with 1.63 hours/0.5781 for CoreSet and 1.42 hours/0.5810 for BADGE. Thus, **at ratio 0.9 on both COCO**

> **and KITTI, AttriSelect provides both better repair performance and lower wall-clock cost on the first repair run, even after charging full-pool VLM assignment**.

CoreSet [3] and BADGE [4] become substantially slower at high source ratios and on larger source pools because their standard greedy procedures update the distance from every one of the N candidates to each new center. Selecting $B = rN$ samples requires $O(NB) = O(rN^2)$ distance-update work at a fixed ratio. The measurements follow this scaling pattern: cost rises sharply with the retained ratio and grows further with the source pool. BADGE can be slower because its gradient embeddings are usually higher-dimensional. AttriSelect does not repeat candidate-to-center computations after its reusable attribute records are available.

Existing Metadata and Later Repair Cycles

Source attributes also need not be assigned from scratch when suitable metadata already exist. For example, the official CelebA release (https://mmlab.ie.cuhk.edu.hk/projects/CelebA.html) provides 40 binary attributes for each of its 202,599 images. **If these public annotations cover the required task-level vocabulary, AttriSelect can use them directly and incurs no VLM assignment cost.** New assignment is needed only for attributes missing from the available metadata.

Cost After Metadata Reuse

The tables are deliberately conservative for AttriSelect because they charge source assignment separately at every ratio. **In repeated repair cycles, the metadata are reused, so the annotation cost is amortized and the remaining online AttriSelect computation is below two seconds.** We will revise Table 2 to label its values explicitly as source-subset selector runtime, add this end-to-end decomposition, and clarify that **the reported 39--283x speedups apply only to source-subset selection after attribute data are available**.

Q2 — Same-Information Attribute-Aware Baselines

> “Comparison look Unfair. AttriSelect is given privileged information like VLM-assigned attributes and explicit bug definitions, but the compared active learning baselines (Entropy, BADGE, CoreSet) do not have access to. The authors should compare it against a baseline with the same information, for example, an ‘Attribute-Filtered Random’ or ‘Attribute-Filtered Entropy’ baseline.”

Response. The generic active-learning baselines do not receive the attribute information used by AttriSelect. They test whether repair-specific metadata adds signal beyond uncertainty, geometric coverage, and gradient embeddings, but they do not test whether our score uses that metadata better than a simple attribute-aware rule.

We added three baselines with access to the same metadata and target-slice definition:

- **Target-Slice + Stratified:** select exact `pink`-and-`fabric` source examples first and fill the remaining budget with class-stratified background samples;
- **Attribute-Filtered Random:** prioritize the `pink`-or-`fabric` source pool and sample randomly within class-balanced queues before filling from the background;
- **Attribute-Filtered Entropy:** use the same candidate pool and class-balanced queues, but rank candidates by predictive entropy from the fixed diagnosis checkpoint.

All methods select only source images and use the same fixed 1,000-image generated set, source budget, Qwen3-VL annotations, class labels, diagnosis checkpoint, generated seed, and 15-epoch repair schedule. The table reports mean overall ACC and standard deviation over seeds 0, 42, 1234, 3407, and 2024. The AttriSelect values are the five-seed results reported in the appendix of the submitted paper.

Source ratio	Target-Slice + Stratified	Attribute-Filtered Random	Attribute-Filtered Entropy	AttriSelect
0.1	0.7303 ± 0.0051	0.7340 ± 0.0034	0.7317 ± 0.0025	**0.7369 ± 0.0043**
0.3	0.7385 ± 0.0024	0.7382 ± 0.0042	0.7365 ± 0.0023	**0.7492** ±

Source ratio	Target-Slice + Stratified	Attribute-Filtered Random	Attribute-Filtered Entropy	AttriSelect
				0.0044
0.5	0.7438 ± 0.0037	0.7442 ± 0.0038	0.7442 ± 0.0031	**0.7508 ± 0.0010**
0.7	0.7434 ± 0.0065	0.7474 ± 0.0038	0.7482 ± 0.0020	**0.7515 ± 0.0037**
0.9	0.7485 ± 0.0026	0.7469 ± 0.0036	0.7503 ± 0.0045	**0.7562 ± 0.0019**

> **Key result.** AttriSelect retains the highest mean ACC at all five source ratios and exceeds the strongest same-information baseline by **0.59 accuracy points on average**. Because all methods receive the same semantic information, this comparison separates the value of attribute access from the additional value of the rarity, generated-data frequency, and soft class-balance score.

Q3 — Interpreting Fix Rate and the Repair Budget

> "Despite the heavy pipeline (VLM tagging, diffusion generation, subset selection), the absolute Fix Rates (FR) in Table 1 remain quite low, mostly ranging between 5% and 13%."

Raw-Metric Interpretation

Response. Fix Rate (FR) should be read together with the original task metric. FR is the **Relative Error Reduction** toward the maximum task score, not the percentage of test examples or failure slices fixed. At the reported ratio-0.9 AttriSelect settings, the raw changes are:

- ImageNet10 accuracy: **0.7173 to 0.7562, or +3.89 percentage points**;
- CelebA accuracy: **0.9057 to 0.9196, or +1.39 percentage points**;
- COCO keypoint AP: **0.5902 to 0.6355, or +4.53 AP**;
- KITTI mAP50--95: **0.5626 to 0.5850, or +2.24 mAP**.

The reported 5--13% range represents a reduction in the model's remaining error, not a 5--13 percentage-point change in the raw task metric. AttriSelect already achieves **6.93% Relative Error Reduction at ImageNet10 ratio 0.1**, raising ACC from 0.7173 to 0.7369 (+1.96 percentage points) while selecting only 10% of the source pool. This is the highest FR at ratio 0.1: the strongest baseline, UHerding, reaches 5.59% FR and 0.7331 ACC. AttriSelect improves FR by **1.34 points (24.0% relative)** and ACC by **0.38 percentage points** under the same budget. Ratio 0.1 is also the most data- and retraining-efficient repair setting evaluated: it uses one-ninth as many selected source examples as ratio 0.9 while retaining the same generated repair set. The fixed source annotation and image-generation costs are accounted for separately in the end-to-end analysis.

Epoch-Budget Sensitivity

As specified in the appendix, the ImageNet10 diagnosis model is trained for 30 epochs at learning rate 1×10^{-4}, while repair uses 15 epochs at 1×10^{-5}. For CelebA, COCO, and KITTI, repair likewise uses half the diagnosis-stage epochs and learning rate 1×10^{-5}. This moderate schedule keeps the comparison focused on source selection rather than the maximum accuracy attainable with longer retraining. Every selector uses the same repair epochs, learning rate, generated set, and source-data budget.

To test whether the reported FR values are mainly limited by this schedule, we additionally vary ImageNet10 repair training from 15 to 60 epochs, corresponding to $0.5\times$--$2\times$ the 30-epoch diagnosis schedule. We evaluate source ratios 0.5 and 0.9 and average over the same five seeds (0, 42, 1234, 3407, and 2024) used in the main paper.

ImageNet10, source ratio 0.5 (FR, %).

Method	**15 epochs ($0.5\times$)**	**30 epochs ($1.0\times$)**	**45 epochs ($1.5\times$)**	**60 epochs ($2.0\times$)**
Random	7.32	9.04	10.04	10.27
Stratified	9.66	9.44	10.49	11.06

Method	15 epochs ($0.5\times$)	30 epochs ($1.0\times$)	45 epochs ($1.5\times$)	60 epochs ($2.0\times$)
Entropy	5.59	10.57	11.08	11.67
CoreSet	8.03	9.99	10.21	11.56
BADGE	10.47	10.74	11.84	12.11
UHerding	8.38	8.94	9.31	10.17
PPAL	8.84	9.29	10.34	10.56
AttriSelect	**11.85**	**12.39**	**13.51**	**14.08**

ImageNet10, source ratio 0.9 (FR, %).

Method	15 epochs ($0.5\times$)	30 epochs ($1.0\times$)	45 epochs ($1.5\times$)	60 epochs ($2.0\times$)
Random	10.22	11.02	12.15	11.53
Stratified	10.47	12.05	12.91	13.22
Entropy	10.22	12.33	13.44	13.76
CoreSet	6.12	12.08	12.31	13.39
BADGE	9.37	10.05	11.09	11.48
UHerding	11.57	12.11	12.37	12.60
PPAL	11.85	12.19	12.81	13.01
AttriSelect	**13.76**	**14.25**	**14.99**	**15.22**

Extending repair training from 15 to 60 epochs raises AttriSelect ACC from 0.7508 to 0.7571 at ratio 0.5 and from 0.7562 to 0.7603 at ratio 0.9. Relative to the pre-repair ACC of 0.7173, the 60-epoch results correspond to gains of 3.98 and 4.30 percentage points, respectively. Over the same schedules, AttriSelect FR increases from 11.85% to 14.08% and from 13.76% to 15.22%. Most baselines also improve overall, although individual trajectories are not strictly monotonic.

> **Key result.** AttriSelect is best in **all eight ratio--epoch configurations**, leading the strongest baseline by 1.38--1.97 FR points at ratio 0.5 and 1.46--1.92 points at ratio 0.9. The persistent ordering supports a selected-set quality effect rather than an early-stopping artifact.

The generation and diagnosis stages are held fixed across source-selection methods, so Table 1 and this additional experiment isolate which source subset gives the best repair result under the same upstream repair data and source budget. We will revise the main results to show raw metrics and absolute changes next to FR and state its interpretation more precisely.

References

1. SafeFix: Targeted Model Repair via Controlled Image Generation. TMLR, 2026.
2. Actively Selecting Annotations Among Objects and Attributes. ICCV, 2011.
3. Active Learning for Convolutional Neural Networks: A Core-Set Approach. ICLR, 2018.
4. Deep Batch Active Learning by Diverse, Uncertain Gradient Lower Bounds. ICLR, 2020.

Response to Reviewer hG6j

Dear Reviewer hG6j, thank you for the questions about the upstream interface, slice granularity, comparison baselines, cost, and vocabulary coverage.

Q1 — Upstream Slice Discovery and Granularity

> “How sensitive is AttriSelect to the upstream slice discovery method and slice granularity?”

Upstream Interface

Response. AttriSelect starts after an upstream debugging method has diagnosed and semantically described a failure slice. As specified in the appendix, the attribute vocabulary, target fields, and VLM assignment prompt are fixed before source selection. Even when the same VLM family contributes to target-slice wording and image-level attribute assignment, the two roles

are separated: the slice description is fixed first, and assignment then runs as a frozen closed-set classifier over source and generated images. The assignment stage does not receive model predictions, failure rates, selector identities, selected-set membership, or downstream repair results.

Under this interface, **two discovery methods that produce the same canonical target conjunction and generated repair set give AttriSelect the same inputs and therefore the same selection result**. The discovery algorithm matters only if it changes the downstream semantic slice, including its fields, target values, granularity, or generated repair data. We will state this condition explicitly: **AttriSelect does not discover or validate the failure slice itself**; it selects source data for an existing debugging and generation pipeline.

Annotation Reliability

The appendix already quantifies the reliability of this semantic interface through a blinded human audit. Five AI graduate students each label 100 CelebA, 100 COCO, and 100 KITTI images using the same closed-set options as the VLM. The annotators do not see VLM labels, target flags, model predictions, data splits, or method outputs. Majority-vote human labels are then compared with the VLM assignments:

Dataset	Target field 1 agreement	Target field 2 agreement	Slice-indicator agreement
CelebA	Hair color: 98%	Skin tone: 99%	92%
COCO	Clothes color: 97%	Weather: 91%	90%
KITTI	Object color: 98%	Weather: 96%	94%

Slice-indicator agreement is the most relevant measure because AttriSelect uses the target conjunction, not either field in isolation. **Agreement is 90--94% across the three audited datasets**, so the selector's semantic slices largely match independent human judgments.

The appendix also corrupts the target-field assignments while keeping the generated repair set, source ratio, and training recipe fixed. At source ratio 0.5, the changes from the clean-VLM

AttriSelect result are:

Dataset	Task metric	5% attribute noise	10% attribute noise	20% attribute noise
CelebA	Test ACC change	0.007	0.01	0.09
COCO	Test AP change	0.002	0.003	0.02
KITTI	Test mAP50--95 change	0.003	0.004	0.06

The small changes under 5% and 10% corruption show that a few isolated assignment errors do not drive AttriSelect. At 20% noise, the larger changes mark the expected failure regime: the attribute counts no longer represent the diagnosed slice reliably. The audit and corruption experiments test the fixed semantic interface. The next experiment instead changes the slice definition.

Slice-Granularity Sensitivity

Slice granularity can still affect repair because it changes both the semantic factors used for source selection and the generated repair set. A coarse slice can mix different failure causes and dilute the relevant attribute signal. A narrow conjunction may have too little support and exclude useful examples that match the main failure factors but not every added condition.

We evaluate this end-to-end sensitivity on COCO and ImageNet10 over five seeds and source ratios 0.1--0.9. For COCO, we compare the diagnosed two-attribute slice `brown AND rainy` with Brown-only, Rain-only, and the narrower three-attribute slice `brown AND rainy AND night`:

Source ratio	Brown-only AP	Rain-only AP	Brown + rainy + night AP	Brown + rainy AP(Ours)
0.1	0.6161	0.6174	0.6097	**0.6183**
0.3	0.6238	0.6224	0.6179	**0.6306**
0.5	0.6284	0.6217	0.6246	**0.6317**

Source ratio	Brown-only AP	Rain-only AP	Brown + rainy + night AP	Brown + rainy AP(Ours)
0.7	0.6327	**0.6333**	0.6291	0.6319
0.9	0.6348	0.6132	0.6336	**0.6355**

The diagnosed two-attribute slice exceeds Brown-only and Rain-only at four of five active ratios, with average gains of **0.24 and 0.80 AP points**, respectively. It exceeds the narrower three-attribute slice at **all five ratios by 0.66 AP points on average**.

For ImageNet10, we compare the diagnosed `pink AND fabric` slice with Pink-only, Fabric-only, and the narrower `pink AND fabric AND rainy background` slice:

Source ratio	Pink-only ACC	Fabric-only ACC	Pink + fabric + rainy ACC	Pink + fabric ACC
0.1	0.7223	0.7346	**0.7398**	0.7369
0.3	0.7369	0.7431	0.7414	**0.7492**
0.5	0.7377	0.7438	0.7429	**0.7508**
0.7	0.7485	0.7423	0.7462	**0.7515**
0.9	0.7500	0.7354	0.7491	**0.7562**

The diagnosed two-attribute slice exceeds both single-attribute variants at all five active ratios, by **0.98 and 0.91 accuracy points on average**, and exceeds the narrower three-attribute slice at four of five ratios by **0.50 points on average**.

> **Granularity takeaway. Downstream performance is relatively insensitive to moderate changes in slice granularity**, although **granularity still matters**: the diagnosed two-attribute definition performs best in **8 of the 10 dataset-ratio settings**. This pattern suggests a support--specificity trade-off: a coarse slice has greater support but includes less relevant examples, whereas an overly narrow conjunction can exclude useful examples.

Q2 — Simple Attribute-Aware Baselines

> “Can the authors compare with simple attribute-aware baselines, such as exact target-slice selection plus stratified sampling?”

Response. We implemented exact target-slice selection with stratified background sampling, along with two additional attribute-aware controls:

- **Target-Slice + Stratified:** select exact `pink`-and-`fabric` source examples first and fill the remaining budget with class-stratified background samples;
- **Attribute-Filtered Random:** prioritize the `pink`-or-`fabric` source pool and sample randomly within class-balanced queues before filling from the background;
- **Attribute-Filtered Entropy:** use the same candidate pool and class-balanced queues, but rank candidates by predictive entropy from the fixed diagnosis checkpoint.

All methods select only source images and use the same fixed 1,000-image generated set, source budget, Qwen3-VL annotations, class labels, diagnosis checkpoint, generated seed, and 15-epoch repair schedule. The table reports mean overall ACC and standard deviation over seeds 0, 42, 1234, 3407, and 2024. The AttriSelect values are the five-seed results reported in the appendix of the submitted paper.

Source ratio	Target-Slice + Stratified	Attribute-Filtered Random	Attribute-Filtered Entropy	AttriSelect
0.1	0.7303 ± 0.0051	0.7340 ± 0.0034	0.7317 ± 0.0025	**0.7369 ± 0.0043**
0.3	0.7385 ± 0.0024	0.7382 ± 0.0042	0.7365 ± 0.0023	**0.7492 ± 0.0044**
0.5	0.7438 ± 0.0037	0.7442 ± 0.0038	0.7442 ± 0.0031	**0.7508 ± 0.0010**
0.7	0.7434 ± 0.0065	0.7474 ± 0.0038	0.7482 ± 0.0020	**0.7515 ± 0.0037**
0.9	0.7485 ± 0.0026	0.7469 ± 0.0036	0.7503 ± 0.0045	**0.7562 ± 0.0019**

> **Key result.** AttriSelect retains the highest mean ACC at all five source ratios and exceeds the strongest same-information baseline by **0.59 accuracy points on average**. Because all methods receive the same semantic information, this comparison separates the value of attribute access from the additional value of the rarity, generated-data frequency, and soft class-balance score.

Q3 — End-to-End Cost Accounting

> "What is the end-to-end cost including generation, VLM attribute assignment, selection, and retraining?"

Accounting Scope

Response. The submitted Table 2 reports subset-selection time, not the cost of the complete repair pipeline. **The sub-second AttriSelect time is selector-only runtime, not an end-to-end speedup** over source annotation, repair-data construction, retraining, and evaluation.

Repair-image generation is a fixed upstream SafeFix [1] dataset-construction stage shared by every selector. Every method receives the same publicly available repair images, so regenerating them from scratch would add the same cost to every method without changing their runtime ordering. We therefore report generation separately rather than charging it only to AttriSelect. In contrast, **we include full source-pool Qwen3-VL assignment for AttriSelect** because it is a method-specific prerequisite.

Conservative Cost of the First Repair Run

Source-pool assignment is a one-time preprocessing cost for a fixed task-level vocabulary. Its metadata can be reused across source ratios and later repair cycles. Here, **the first repair run assumes that source attributes must be assigned from scratch**. Its cost covers source preprocessing and selection through retraining and evaluation; repair-image generation is reported separately. The tables conservatively charge a fresh full-pool assignment in every AttriSelect row. For KITTI, the mean measured

assignment time over the two complete 5,000-image runs is **32.77 minutes** on a single NVIDIA H200 GPU (the previously reported 10--20 minutes per 1,000 images was measured on an NVIDIA A5000, whereas an H200 requires only approximately 5 minutes per 1,000 images). On the same GPU, assigning attributes to the complete 20,000-image COCO source pool takes approximately **1.50 hours**.

KITTI: 5,000 source images. Each AttriSelect row charges the mean fresh 5,000-image H200 Qwen3-VL assignment time of 32.77 minutes.

Ratio	Random	Stratified	Entropy	CoreSet	BADGE	UHerding	
0.1	42.17 min	42.18 min	48.82 min	1.37 h	1.12 h	43.47 min	43.5
0.3	43.84 min	43.85 min	50.28 min	1.44 h	1.20 h	45.08 min	45.6
0.5	45.18 min	45.18 min	51.73 min	1.49 h	1.27 h	46.42 min	47.4
0.7	47.04 min	47.05 min	53.46 min	1.56 h	1.35 h	48.16 min	49.7
0.9	48.86 min	48.87 min	55.53 min	1.63 h	1.42 h	50.34 min	51.4

COCO: 20,000 source images. Image generation and generated-repair-image assignment are excluded. Each entry includes the applicable source preprocessing and selection cost, retraining, and evaluation. The AttriSelect column charges a fresh full-pool Qwen3-VL assignment in every row.

Ratio	Random	Stratified	Entropy	CoreSet	BADGE	UHerding	
0.1	4.41 min	4.42 min	10.98 min	16.18 min	19.63 min	5.08 min	
0.3	8.18 min	8.18 min	14.74 min	39.86 min	49.76 min	8.93 min	
0.5	12.37 min	12.36 min	19.04 min	1.08 h	1.56 h	13.08 min	
0.7	15.73 min	15.75 min	22.27 min	1.45 h	2.29 h	16.34 min	
0.9	19.66 min	19.65 min	26.28 min	1.87 h	3.01 h	20.18 min	

> **Cost takeaway.** At small source ratios, lightweight conventional selectors have lower wall-clock cost on the first repair run. At ratio 0.9 on both COCO and KITTI, however, **AttriSelect**

> **is better than CoreSet and BADGE in both repair performance and first-run wall-clock cost**.

On COCO, AttriSelect takes 1.72 hours and reaches 0.6355 AP, compared with 1.87 hours/0.6202 for CoreSet and 3.01 hours/0.6213 for BADGE. On KITTI, it takes 1.15 hours and reaches 0.5850 mAP50--95, compared with 1.63 hours/0.5781 and 1.42 hours/0.5810.

Existing Metadata and Later Repair Cycles

Source attributes need not be assigned from scratch when suitable metadata already exist. For example, the official CelebA release (https://mmlab.ie.cuhk.edu.hk/projects/CelebA.html) provides 40 binary attributes for each of its 202,599 images. **If these public annotations cover the required task-level vocabulary, AttriSelect can use them directly and incurs no VLM assignment cost.** New assignment is needed only for attributes missing from the available metadata.

Cost After Metadata Reuse

The tables are deliberately conservative because they charge source assignment again at every ratio. **In repeated repair cycles, the metadata are reused, so this annotation cost is amortized and the remaining online AttriSelect computation is below two seconds.** We will revise Table 2 to label its values explicitly as source-subset selector runtime and distinguish one-time preprocessing, the first repair run, and later runs with reused metadata.

Q4 — Incomplete Attribute-Vocabulary Coverage

> "How does the method behave when the failure slice is not well captured by the closed-set attribute vocabulary?"

Response. This question concerns coverage of the attribute schema, not synonymous wording. Once names and values are mapped to canonical closed-set labels, changing their surface form does not change the AttriSelect score. The two substantive failure cases are a missing factor and a category that merges distinct values too coarsely.

If a target factor is absent, AttriSelect cannot explicitly prioritize that factor. It can use only the remaining observed attributes and their correlations with the failure slice. The repair-specific part of the score therefore weakens, while soft class balance still preserves label coverage. **If target and non-target values are merged, class-conditional rarity and the generated-data frequency penalty become less discriminative**, which can cause the selected subset to include more examples outside the true fine-grained slice. These are limitations of the current closed-set formulation.

We ran controlled missing-field studies on ImageNet10 and COCO. For each dataset, we keep the generated repair pool, generated-image budget, source budget, checkpoint, and training schedule fixed, and remove one target field from the source-selection vocabulary and its corresponding generated-count term. `Full` uses both target fields for source selection. In each missing-field cell, the value outside parentheses is the downstream task metric after retraining.

ImageNet10: target `pink AND fabric`, overall test accuracy (%). Values in parentheses `(...)` are the selected-set Jaccard overlap with `Full` at the same ratio.

Ratio	Full	Missing color	Missing texture
0.1	73.69	72.69 (0.5604)	**73.85** (0.3904)
0.3	**74.92**	73.77 (0.6166)	73.54 (0.5842)
0.5	**75.08**	74.23 (0.6443)	74.23 (0.7330)
0.7	**75.15**	74.00 (0.6999)	74.23 (0.8826)
0.9	**75.62**	75.08 (0.8765)	74.54 (0.9668)

COCO: target `brown AND rainy`, keypoint AP. Values in parentheses `(...)` are the selected-set Jaccard overlap with `Full` at the same ratio.

Ratio	Full	Missing color	Missing weather
0.1	**0.6183**	0.6055 (0.4782)	0.6105 (0.2034)

Ratio	Full	Missing color	Missing weather
0.3	**0.6306**	0.6221 (0.5611)	0.6214 (0.4613)
0.5	**0.6317**	0.6178 (0.7063)	0.6271 (0.6084)
0.7	0.6319	**0.6332** (0.7836)	0.6323 (0.8110)
0.9	**0.6355**	0.6237 (0.8969)	0.6244 (0.8708)

> **Vocabulary-coverage takeaway.** Missing a target field changes the selected source set most strongly at low budgets: at ratio `0.1`, Jaccard falls to `0.5604/0.3904` for missing color/texture on ImageNet10 and `0.4782/0.2034` for missing color/weather on COCO. Downstream changes are non-monotonic, however. Mean ImageNet10 accuracy changes are `-0.94/-0.81` points for missing color/texture, while mean COCO AP changes are `-0.0091/-0.0065` for missing color/weather. **Missing-field variants are lower than Full in 17 of the 20 dataset--field--ratio comparisons.** Incomplete coverage therefore changes source selection and generally lowers task performance, although the effect is not strictly monotonic at every ratio.

This setting differs from the submission's attribute-noise experiment, which corrupts image-level assignments while retaining the correct fields and candidate values. The present study tests absent fields; the separate merged-value transformation remains unrun, so we do not claim robustness to overly coarse value categories.

References

1. SafeFix: Targeted Model Repair via Controlled Image Generation. TMLR, 2026.

Response to Reviewer KaLa

Dear Reviewer KaLa, thank you for the questions about the selected data pool, repair protocol, and compatibility with other pipelines.

W1/Q1 — Selected Data Pool

> “The problem formulation needs to be clarified. It is not always clear whether the proposed method is selecting generated repair data, original source data, or a combination of both. The descriptions in the abstract, introduction, and conclusion appear somewhat vague or potentially inconsistent on this point. The authors are encouraged to state explicitly what data pool AttriSelect operates on and how the selected data are used during retraining.”

Response. AttriSelect [1] selects only from the original source or training pool; it does not select among the generated repair images. In every main experiment, **the generated set is fixed to 1,000 images for all selectors**. The retraining data are

$$\mathcal{D}_{\text{repair}} = \mathcal{D}_{\text{sel}}^{\text{src}} \cup \mathcal{D}_{\text{gen}},$$

where AttriSelect chooses only $\mathcal{D}_{\text{sel}}^{\text{src}} \subseteq \mathcal{D}_{\text{src}}$. The generated set affects source selection through its attribute-frequency counts, but its membership remains fixed. We will state this directly in the abstract, introduction, framework figure, and conclusion, and remove notation that suggests a separately selected generated subset.

W2/Q4 — Prior-Work Taxonomy

> “The categorization of prior model repair methods requires further clarification. In particular, reference [7] seems to be associated with both the first and second lines of work, which makes the taxonomy confusing. The authors should revise this discussion to make the distinctions among different categories more precise.”

Response. The repeated citation is intentional: HiBug spans two stages of the repair pipeline by discovering attribute-defined failure slices and generating repair examples for them.

We will organize the revision by pipeline stage instead of treating the categories as mutually exclusive method groups. The stages are **failure-slice discovery, repair-data construction, source-data selection, and model retraining**. HiBug can then appear in more than one stage without

making the taxonomy inconsistent.

W3/Q2 — Repair-Pipeline Implementation

> “It is unclear what repair method or methods are used together with AttriSelect in the experiments. Since AttriSelect is a data selection strategy rather than a complete repair algorithm, the experimental section should clearly specify the underlying repair pipeline, retraining/fine-tuning procedure, and how the selected source samples are combined with the generated repair images.”

Response. All source-selection methods use the same protocol. We first train or obtain the diagnosis checkpoint θ_t, diagnose a semantic failure slice, and construct a fixed generated repair set. Each method selects a source subset at the same ratio. We combine that subset with the fixed generated images and continue training or fine-tune θ_t with the same optimizer, learning-rate change from 1e-4 to 1e-5, epoch count, and generated budget. **The selector is the only component that changes within each dataset-level comparison.**

These details are currently split across several sections. We will consolidate them in a table listing each dataset's diagnosed slice, source pool, generator, generated budget, base checkpoint, model, and repair schedule.

TODO: More details, implementation

W4/Q3 — Compatibility with Alternative Pipelines

> “The paper states that AttriSelect is best viewed as one component in a localized repair pipeline. However, the experiments do not appear to evaluate its compatibility with multiple repair pipelines. It would strengthen the paper if the authors could test AttriSelect with different repair methods or, at least, discuss the extent to which the conclusions depend on the specific repair pipeline used in the experiments.”

> Additional point in Q3: “Could the authors provide evidence or discussion about whether AttriSelect remains effective when paired with different [...] retraining strategies?”

Alternative-Generator Evidence

Response. AttriSelect is a source-selection component, not a complete repair pipeline. It requires a diagnosed semantic slice, an annotated source pool, and a fixed generated repair set. The appendix tests whether the result depends on the main ControlNet generator. We replace ControlNet with InstructPix2Pix on CelebA and ICEdit on COCO while holding the selectors, generated-image budget, source ratios, and repair protocol fixed.

CelebA with InstructPix2Pix generation. Metric: FR (%).

Ratio	Random	Stratified	Entropy	CoreSet	BADGE	UHerding	PPAL	(Instru
0.1	4.23	1.95	6.41	-6.65	2.34	-1.78	5.12	
0.3	5.34	4.72	7.98	7.21	1.02	2.81	3.45	
0.5	5.56	7.23	6.78	7.54	2.12	1.56	9.12	
0.7	2.12	7.67	9.34	5.78	7.34	4.23	5.34	
0.9	2.45	3.92	7.56	5.34	5.23	3.87	5.91	

> **Key result.** AttriSelect is the strongest source selector at every active CelebA ratio under InstructPix2Pix, exceeding the strongest baseline by **2.51 FR points on average**.

COCO pose estimation with ICEdit generation. Metric: FR (%).

Ratio	Random	Stratified	Entropy	CoreSet	BADGE	UHerding	PPAL	Ou (ICEd
0.1	3.78	5.23	5.87	-0.34	2.34	3.12	1.56	**7.**
0.3	3.92	8.67	8.12	9.12	6.56	4.67	6.89	**10.**
0.5	7.91	8.45	9.67	6.91	7.82	6.54	7.23	**10.**

Ratio	Random	Stratified	Entropy	CoreSet	BADGE	UHerding	PPAL	Ou (ICEd
0.7	6.54	10.12	9.98	9.56	9.12	6.71	8.23	**10.8**
0.9	5.12	9.87	6.98	7.82	8.12	7.54	10.12	**11.5**

> **Key result.** AttriSelect is also the strongest source selector at every active COCO ratio under ICEdit, exceeding the strongest baseline by **1.13 FR points on average**. Across both alternative generators, AttriSelect ranks first in **all 10 active dataset-ratio settings**; **the selection benefit is not specific to ControlNet**.

Scope of the Evidence

> **Scope.** These experiments establish compatibility with multiple generators. They do not establish invariance to every repair algorithm or fine-tuning strategy, which remains future work.

W5 — Reference Formatting

> “The formatting of the references should be improved. For example, the names of journals and conferences should be made consistent, either using full names or standard abbreviations throughout the reference list.”

Response. We thank the reviewer for catching this inconsistency. **We have standardized conference and journal names throughout the reference list** using standard abbreviations, including NeurIPS, ICLR, ICML, CVPR, ICCV, ECCV, WACV, AAAI, FAccT, and JMLR. We also corrected inconsistent capitalization in venue and title fields, protected proper names such as ImageNet, COCO, KITTI, and SafeFix, and fixed a malformed author field in the PPAL reference. The revised bibliography now follows one format throughout.

References

1. SafeFix: Targeted Model Repair via Controlled Image Generation. TMLR, 2026.

2. Actively Selecting Annotations Among Objects and Attributes. ICCV, 2011.

Misc

Complexity Derivation for CoreSet and BADGE

Let N denote the number of source candidates, $B = rN$ the requested subset size at source ratio r, d the feature dimension used by CoreSet, and D the gradient-embedding dimension used by BADGE. The following argument concerns the standard greedy procedures in the original methods; model inference or feature extraction adds a separate term that is at least linear in N.

CoreSet applies greedy farthest-first traversal to approximate the k-center objective~\citep{sener2018active}. After t samples have been selected, it maintains each candidate's distance to its nearest selected center,

$$\Delta_i^{(t)} = \min_{j \in S_t} \|z_i - z_j\|_2^2,$$

selects $j_t = \arg\max_i \Delta_i^{(t)}$, and performs the update

$$\Delta_i^{(t+1)} = \min\left\{\Delta_i^{(t)}, \|z_i - z_{j_t}\|_2^2\right\} \qquad \text{for every } i \in \{1, \dots, N\}.$$

Each iteration therefore computes N distances in d dimensions and costs $\Theta(Nd)$. Repeating this update for B selected centers gives

$$T_{\mathrm{CoreSet}}(N, B, d) = \Theta(NBd) = \Theta(rN^2 d).$$

An incremental implementation requires $\Theta(Nd + N)$ memory for the embeddings and nearest-center distances; an implementation that materializes all pairwise distances instead requires $\Theta(N^2)$ memory. Thus, even the memory-efficient implementation retains the quadratic-in-N dominant computation when the selection ratio is fixed.

BADGE first constructs a gradient embedding $g_i \in \mathbb{R}^D$ for every candidate and then applies k -means++ seeding to select a diverse batch~\citep{ash2020deep}. Its seeding step similarly maintains

$$\delta_i^{(t)} = \min_{j \in S_t} \|g_i - g_j\|_2^2.$$

Adding one center updates this quantity for all N candidates at a cost of $\Theta(ND)$ per iteration. Therefore,

$$T_{\mathrm{BADGE}}(N, B, D) = \Theta(ND) + \Theta(NBD) = \Theta(rN^2D),$$

where the first term constructs or stores the gradient embeddings and the second is the diversity-seeding cost. For last-layer gradient embeddings, D usually scales with the feature dimension times the number of output classes, so it can be much larger than CoreSet's d. BADGE also requires $\Theta(ND + N)$ memory under incremental distance updates. These expressions describe the scaling trend rather than exact wall-clock time; GPU utilization, batching, task heads, and implementation details affect the constants.

The measurements show both effects predicted by the derivation. For fixed N, increasing the ratio increases B, so the dominant selection work grows approximately linearly in r. In the original COCO timings, CoreSet takes 5, 15, and 25 minutes as r increases from 0.1 to 0.3 to 0.5; BADGE takes 21 minutes, 1 hour, and 2 hours. For fixed r, increasing the source pool by a factor c increases the dominant NB term by c^2. CelebA has four times as many source images as COCO (80,000 versus 20,000), so its asymptotic distance-update work can be $4^2 = 16$ times larger before accounting for the embedding dimension and model cost. This scaling moves standard CoreSet and BADGE implementations from minutes on COCO to hours on CelebA. AttriSelect does not perform an $N \times B$ sequence of embedding-distance updates after its reusable attribute records are available.

Reviewer hG6j

Q1

We exclude ratio 1.0 because every method retains the complete source pool at that ratio. The

variants also use their corresponding generated repair pools, so this experiment measures the combined effect of slice specification, source-selection vocabulary, and repair-data construction; **it does not isolate selector granularity while holding generated images fixed**.